

[DIS 1] ReLUs and Neural Network Intuition

If you are running on a local anaconda install, you will need to install pytorch with the command

```
conda install pytorch -c pytorch
```

If you are running on Colab, run the cell below and ensure that `helpers.py q_relu_vis.ipynb` is outputted

```
!git clone https://github.com/Berkeley-CS182/cs182fa26_public.git
%cd cs182fa26_public/dis01/code
!ls

Cloning into 'cs182fa26_public'...
remote: Enumerating objects: 1119, done.
remote: Counting objects: 100% (219/219), done.
remote: Compressing objects: 100% (131/131), done.
remote: Total 1119 (delta 97), reused 153 (delta 66), pack-reused 900 (from 2)
Receiving objects: 100% (1119/1119), 209.20 MiB | 19.18 MiB/s, done.
Resolving deltas: 100% (480/480), done.
Updating files: 100% (683/683), done.
/content/cs182fa26_public/dis01/code/cs182fa26_public/dis01/code
helpers.py q_relu_vis.ipynb
```

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
import copy
import time
from ipywidgets import fixed, interactive, widgets

from helpers import *

%matplotlib inline
```

Generate Training and Test Data

We are using piecewise linear function. Our training data has added noise $y = f(x) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$. The test data is noise free.

Once you have gone through the discussion once you may wish to adjust the number of training samples and noise variance to see how gradient descent behaves under the new conditions.

```
f_type = 'piecewise_linear'

def f_true(X, f_type):
    if f_type == 'sin(20x)':
        return np.sin(20 * X[:,0])
    else:
        TenX = 10 * X[:,0]
        _ = 12345
        return (TenX - np.floor(TenX)) * np.sin(_ * np.ceil(TenX)) - (TenX - np.ceil(TenX)) * np.sin(_ * np.floor(TenX))

n_features = 1
n_samples = 200
sigma = 0.1
rng = np.random.RandomState(1)

# Generate train data
X = np.sort(rng.rand(n_samples, n_features), axis=0)
y = f_true(X, f_type) + rng.randn(n_samples) * sigma

# Generate NOISELESS test data
X_test = np.concatenate([X.copy(), np.expand_dims(np.linspace(0., 1., 1000), axis=1)])
X_test = np.sort(X_test, axis=0)
y_test = f_true(X_test, f_type)
```

Define the Neural Networks

We will learn the piecewise linear target function using a simple 1-hidden layer neural network with ReLU non-linearity, defined by

$$\hat{y} = \mathbf{W}^{(2)} \Phi \left(\mathbf{W}^{(1)} x + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)}$$

where $\Phi(x) = \text{ReLU}(x)$ and superscripts refer to indices, not the power operator.

We will also create two SGD optimizers to allow us to choose whether to train all parameters or only the linear output layer's parameters.

Note that we use separate learning rates for the two version of training. There is too much variance in the gradients when training all layers to use a large learning rate, so we have to decrease it.

We will modify the default initialization of the biases so that the ReLU elbows are all inside the region we are interested in.

We create several versions of this network with varying widths to explore how hidden layer width impacts learning performance.

Once you have gone through the discussion once you may wish to train networks with even larger widths to see how they behave under the three different training paradigms in this notebook.

```
# Don't rerun this cell after training or you will lose all your work
nets_by_size = {}
```

```
widths = [10, 20, 40]
for width in widths:
    # Define a 1-hidden layer ReLU nonlinearity network
    net = nn.Sequential(nn.Linear(1, width),
                        nn.ReLU(),
                        nn.Linear(width, 1))

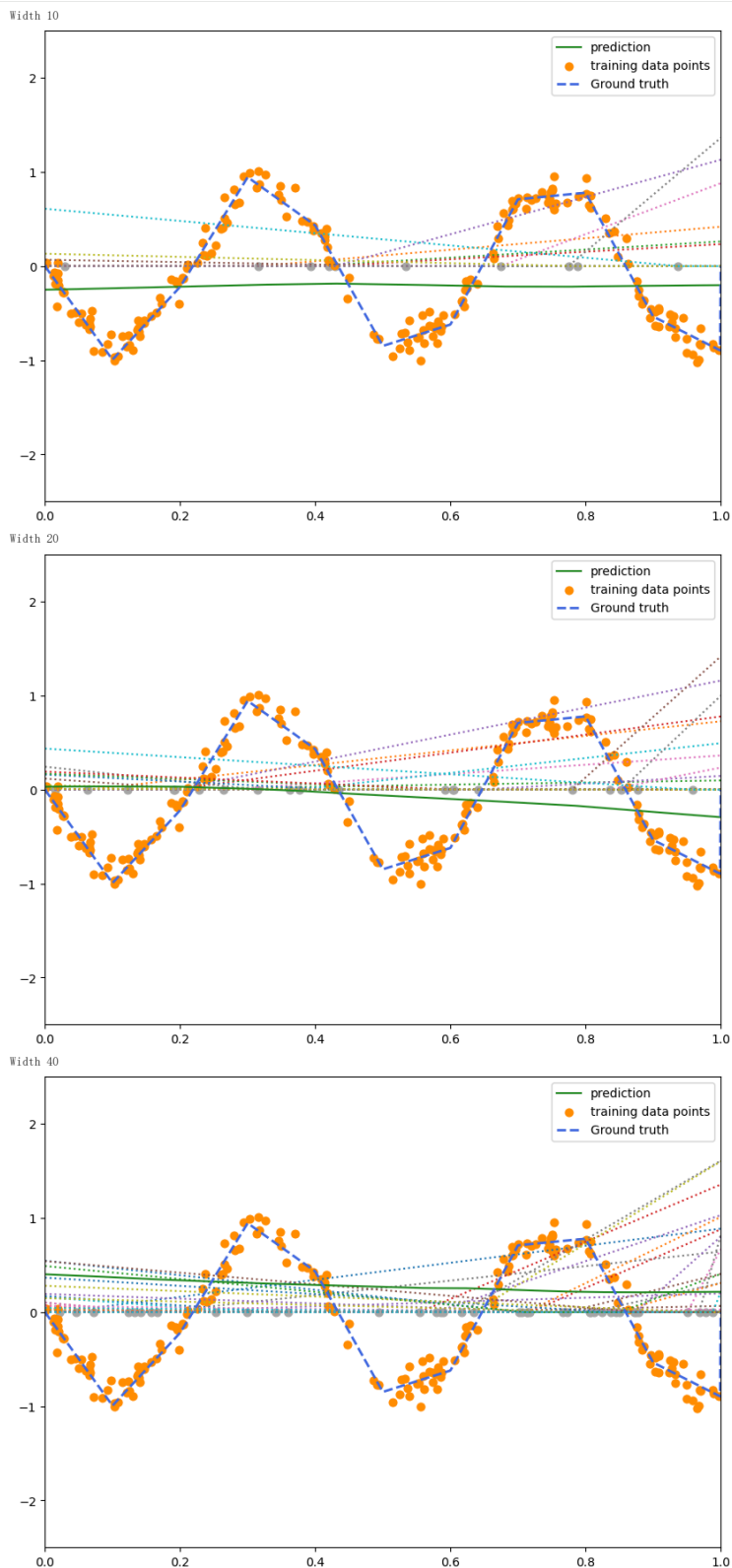
    loss = nn.MSELoss()
    # Get trainable parameters
    weights_all = list(net.parameters())
    # Get the output weights alone
    weights_out = weights_all[2:]
    # Adjust initial biases so elbows are in [0,1]
    elbows = np.sort(np.random.rand(width))
    print("Elbows located at:")
    print(elbows)
    new_biases = -elbows * to_numpy(weights_all[0]).ravel()
    weights_all[1].data = to_torch(new_biases)
    # Create SGD optimizers for outputs alone and for all weights
    lr_out = 0.2
    lr_all = 0.02
    opt_all = torch.optim.SGD(params=weights_all, lr=lr_all)
    opt_out = torch.optim.SGD(params=weights_out, lr=lr_out)
    # Save initial state for comparisons
    initial_weights = copy.deepcopy(net.state_dict())
```

```
# print("Initial Weights", initial_weights)
nets_by_size[width] = {'net': net, 'opt_all': opt_all,
                        'opt_out': opt_out, 'init': initial_weights}
```

Visualize the starting ReLU corners & slopes, and total function

The dashed lines represent the output of an individual ReLU function in the hidden layer for an x input passed through the first linear layer. The gray points mark the elbows of the ReLU functions where their output becomes non-zero.

```
for w in widths:
    net = nets_by_size[w]['net']
    print("Width", w)
    plot_update(X, y, X_test, y_test, net)
```



Train all the networks now - this will take a while!

You can expect training to take between 5 and 10 minutes depending on whether you run locally or on Colab and how heavily loaded Colab is at the moment.

Train only the output layer

```

n_steps = 150000
save_every = 1000
t0 = time.time()
for w in widths:
    print("-"*40)
    print("Width", w)
    net = nets_by_size[w]['net']
    opt_out = nets_by_size[w]['opt_out']
    initial_weights = nets_by_size[w]['init']
    history_output = train_network(X, y, X_test, y_test,
                                   net, optim=opt_out,
                                   n_steps=n_steps, save_every=save_every,
                                   initial_weights=initial_weights,
                                   verbose=False)

    nets_by_size[w]['hist_out'] = history_output
    print("Width", w)
    plot_test_train_errors(history_output)
    print("Elapsed time %.1f minutes" % ((time.time() - t0) / 60))

t1 = time.time()
print("-"*40)
print("Trained output layer in %.1f minutes" % ((t1 - t0) / 60))

```

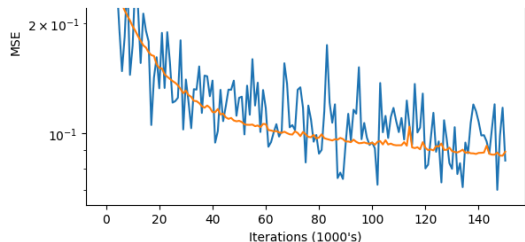

Width 10
Train all layers

```
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000

n_steps = 150000
save_every = 1000
t0 = time.time()
for w in widths:
    print("-"*40)
    print("Width", w)
    net = nets_by_size[w]['net']
    opt_all = nets_by_size[w]['opt_all']
    initial_weights = nets_by_size[w]['init']
    history_all = train_network(X, y, X_test, y_test,
                               net, opt=opt_all,
                               n_steps=n_steps, save_every=save_every,
                               initial_weights=initial_weights,
                               verbose=False)

    nets_by_size[w]['hist_all'] = history_all
    print("Width", w)
    plot_test_train_errors(history_all)

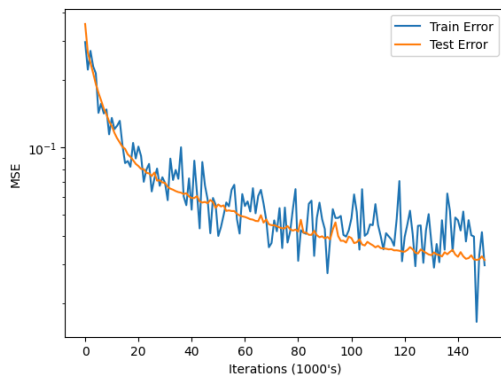
t1 = time.time()
print("-"*40)
print("Trained all layers in %.1f minutes" % ((t1 - t0) / 60))
```



Elapsed time 2.0 minutes

Width 20
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000

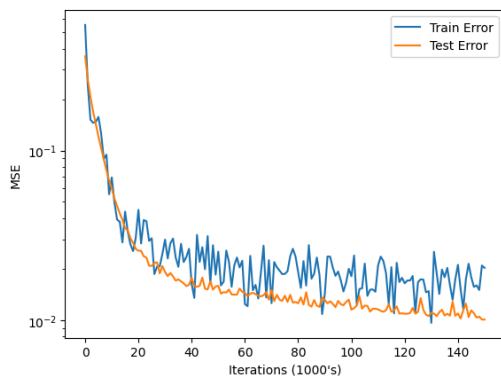
Width 20



Elapsed time 3.9 minutes

Width 40
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000

Width 40



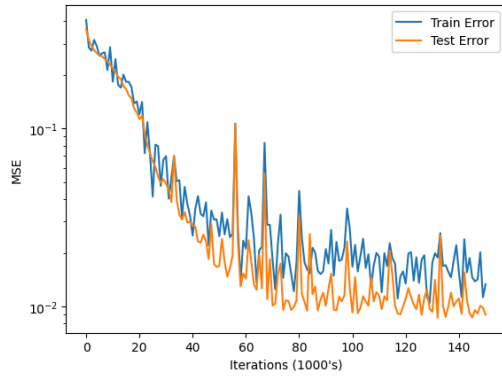
Elapsed time 5.8 minutes

Trained output layer in 5.5 minutes

Width 10

SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000

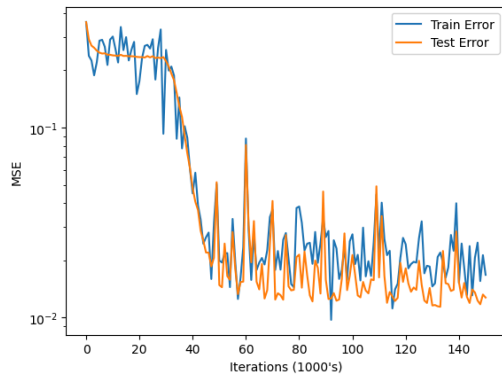
Width 10



Width 20

SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000

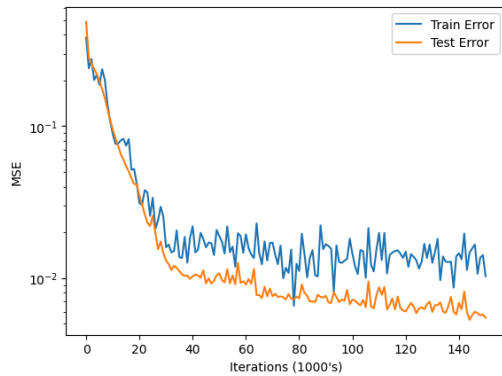
Width 20



Width 40

SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000

Width 40



Trained all layers in 5.5 minutes

Train and Test Error

In the train and test error plots, you should observe that, after sufficient training, the test error is consistently lower than the train error. Normally we expect the opposite to happen. **What is causing test error to be lower than train error?**

The training data contains noise, while the test data are noiseless.

Learn the Output Layer Using Ridge Regression

We can treat the output of the hidden layer as a featurization of x and use ridge regression to choose weights for the output layer instead of iterating with SGD. We have set up the infrastructure to extract the featurization of the training data and write your learned coefficients back into the network. You will **perform ridge regression with the hidden layer featurization with your choice of regularization parameter λ** and compare the result with networks learned by gradient descent.

Learn the last layer weights with ridge regression for each network width and compare the shape of the learned functions, test error, and best regularization coefficients.

You don't need to rigorously choose the regularization coefficient through cross-validation, but you should at least explore a range of values and pick a reasonable value.

```
width = 40 # Options are 10, 20, 40
# Reinitialize net
net = nets_by_size[width]['net']
initial_weights = nets_by_size[width]['init']
net.load_state_dict(initial_weights)

# Featurize x using the hidden layer
Wl = initial_weights['0.weight']
b1 = initial_weights['0.bias']
Xnew = to_torch(X) @ Wl.T + b1[None, :]
Xnew = to_numpy(nn.functional.relu(Xnew))
# Add the bias term
tmp = np.ones((Xnew.shape[0], width + 1))
tmp[:, :-1] = Xnew
Xnew = tmp

# TODO: Perform ridge regression on the featurization of x
# Store the learned coefficients in a width+1 size array called 'coeffs'
### start last_layer_rr ###
lambda = 1e-6
# w = (X^T X + lambda I)^-1 X^T y
#hint: use np.linalg.solve instead of np.linalg.inv
reg = np.eye(width + 1)
coeffs = np.linalg.solve(
    Xnew.T @ Xnew + lambda * reg,
    Xnew.T @ y
)
print("Learned coeffs:", coeffs)
### end last_layer_rr ###
```